AGENTS

AGENTS.md

> Base agent rules: `constitution/AGENTS.md` — READ IT FIRST. The base file is authoritative for any topic not covered here. Project-specific rules below extend them; they never weaken them.
>
> **Propagated clauses:** §11.4.10, §11.4.24, §11.4.30, §11.4.44, §11.4.65, §11.4.66, §11.4.75, §11.4.78, §11.4.85, §11.4.109, §11.4.113, §11.4.125

Compact instruction file for AI agents working in this repo. For deeper narrative docs see `CLAUDE.md`, `docs/USER_MANUAL.md`, `docs/PLUGINS.md`, `docs/SECURITY.md`.

What This Project Is

qBittorrent enhancement: multi-tracker search, authenticated download proxy, and a Merge Search Service. **Not a Python package** – no installable distribution. Runtime is container-based. Config is unified in `pyproject.toml`. A Go/Gin backend is available as an opt-in replacement (`--profile go`).

Jackett is fully auto-configured: `start.sh` starts the Jackett container, waits for it to generate its API key, extracts it from `config/jackett/Jackett/ServerConfig.json`, and injects it into the proxy containers via `JACKETT_API_KEY`. No manual configuration is required. See `docs/JACKETT_INTEGRATION.md` for details.

The project provides: - **Merge Search Service** (FastAPI on `:7187`) – fans out searches across 40+ public and private trackers, deduplicates and enriches results, streams real-time updates via SSE. - **Download Proxy** (Python HTTP server on `:7186`) – proxies qBittorrent WebUI and handles authenticated downloads for private trackers. - **WebUI Bridge** (Python HTTP server on `:7188`) – host process that bridges the

qBittorrent WebUI with private-tracker authentication (RuTracker, Kinozal, NNM-Club, IPTorrents). - **Angular 21 Dashboard** – standalone SPA frontend served from the merge service, distinct from the FastAPI Jinja2 dashboard.

Technology Stack

Layer	Technology	Version / Notes
Python runtime	CPython	>=3.12 (target 3.12, CI also tests 3.13)
Python web framework	FastAPI + uvicorn	Async handlers, SSE streaming
Go runtime	Go	1.26.2
Go web framework	Gin	v1.12.0
Frontend framework	Angular	21.x (signals-based, standalone components)
Frontend test runner	Vitest	Via `@angular/build` / `ng test`
Frontend package manager	npm	10.9.3 (locked in `packageManager`)
Container runtime	Podman (preferred) or Docker	Auto-detected in all shell scripts
Orchestration	docker-compose / podman compose	`network_mode: host`
qBittorrent image	`lscr.io/linuxserver/qbittorrent:latest`	Internal WebUI on `:7185`
Jackett image	`lscr.io/linuxserver/jackett:latest`	Port `:9117`, auto-configured

Key Python runtime dependencies (`download-proxy/requirements.txt`): `fastapi`, `uvicorn`, `aiohttp`, `pydantic`, `Levenshtein`, `cachetools`, `filelock`, `tenacity`, `pybreaker`, `requests`, `urllib3`.

Key test dependencies (`tests/requirements.txt`): `pytest`, `pytest-asyncio`, `pytest-cov`, `pytest-timeout`, `pytest-randomly`, `pytest-xdist`, `pytest-docker`, `pytest-benchmark`, `hypothesis`, `responses`, `respx`, `freezegun`, `schemathesis`, `playwright`, `locust`, `ruff`, `mypy`, `bandit`, `pip-audit`, `mutmut`.

Architecture

Two-container setup via `docker-compose.yml` (`network_mode: host`), with an optional Go backend and a host-level bridge process:

Service	Container / Process	Image / Binary	Ports	Notes
qBittorrent	`qbittorrent`	`lscr.io/linuxserver/qbittorrent:latest`	7185	Internal WebUI; hardcoded credentials `admin`/`admin`
Jackett	`jackett`	`lscr.io/linuxserver/jackett:latest`	9117	Auto-configured; API key extracted at startup
Download proxy + Merge service	`qbittorrent-proxy`	`python:3.12-alpine`	7186, 7187	Python multi-threaded entrypoint (`download-proxy/src/main.py`)
Go backend	`qbittorrent-proxy-go`	Built from `qBitTorrent-go/Dockerfile`	7186, 7187, 7188	Opt-in via `--profile go`; replaces Python proxy
boba-jackett	`boba-jackett`	Built from `qBitTorrent-go/Dockerfile.jackett`	7189	Owns Jackett credentials + indexer overrides + autoconfig

Service	Container / Process	Image / Binary	Ports	Notes
				run history; backed by encrypted SQLite at `/config/boba.db`
WebUI bridge	Host process	`python3 webui-bridge.py`	7188	Not a container; bridges private-tracker auth

Port map: - **7185** – qBittorrent WebUI (container-internal) - **7186** – Download proxy -> qBittorrent - **7187** – Merge Search Service (FastAPI + Angular SPA + Jinja2 dashboard) - **7188** – webui-bridge (private-tracker downloads) - **7189** – boba-jackett (Go) — `http://localhost:7189` - **9117** – Jackett health/API endpoint

Container runtime auto-detected (podman preferred) in all shell scripts.

Python entrypoint (`download-proxy/src/main.py`)

Starts two daemon threads: 1. **Original proxy** – imports and runs `download_proxy.py` (from the engines dir) for legacy proxy support on `:7186`. 2. **FastAPI server** – runs `api:app` via uvicorn on `:7187`.

Signal handlers for `SIGTERM`/`SIGINT` trigger a graceful shutdown via `threading.Event`.

Merge Search Service internals

The search orchestrator (`merge_service/search.py`) is the central coordinator.

Concurrency controls: - `MAX_CONCURRENT_SEARCHES` (default 8) – caps in-flight fan-out tasks; returns HTTP 429 when saturated. - `MAX_CONCURRENT_TRACKERS` (default 5) – per-search semaphore limiting

parallel tracker subprocesses/aiohttp sessions. - `PUBLIC_TRACKER_DEADLINE_SECONDS` (default 60, clamped 5–120) – kills unresponsive public tracker subprocesses.

Fan-out flow: 1. `start_search()` creates a `SearchMetadata` with pre-seeded `tracker_stats` so the dashboard knows which trackers will be hit. 2. `_run_search()` acquires the tracker semaphore, then calls `_search_tracker()` for each enabled source. 3. **Public trackers** run in isolated subprocesses (`asyncio.create_subprocess_exec`) executing a dynamically generated Python script that patches `novaprinter.prettyPrinter` to emit NDJSON lines. The orchestrator reads stdout line-by-line with a deadline so partial results are preserved even on timeout. 4. **Private trackers** use direct `aiohttp` with session cookies (RuTracker, Kinozal, NNMClub, IPTorrents). 5. Per-tracker `TrackerSearchStat` objects update in real time (status, duration, error classification, authentication flag). 6. When all trackers finish, `Deduplicator.merge_results()` runs once, metadata flips to `completed`.

Error classification: `_classify_plugin_stderr()` categorizes subprocess stderr into structured diagnostics (`upstream_http_403`, `dns_failure`, `plugin_parse_failure`, `deadline_timeout`, etc.).

Deduplication (`merge_service/deduplicator.py`) uses a tiered matching engine: metadata identity -> infohash -> name+size -> fuzzy Levenshtein.

Enrichment (`merge_service/enricher.py`) resolves titles against TMDB, OMDb, TVMaze, AniList, MusicBrainz, and OpenLibrary to add posters, years, genres, and content types.

Validation (`merge_service/validator.py`) performs HTTP (BEP 48) and UDP (BEP 15) scrape health checks with a bencode parser.

Data stores are all TTL-bounded (`cachetools.TTLCache`) to prevent memory leaks: `_active_searches`, `_tracker_results`, `_last_merged_results`, `_tracker_sessions`.

Go backend (`qBitTorrent-go/`)

- `cmd/qbittorrent-proxy/` – main API binary serving `:7187`.
- `cmd/webui-bridge/` – bridge binary serving `:7188`.
- `internal/api/` – HTTP handlers (health, search, hooks, download, scheduler, theme).
- `internal/service/` – Merge search orchestrator + SSE broker.

- `internal/client/` – qBittorrent Web API client (auth, search, torrents).
- `internal/models/` – Data types (auth, hook, scheduler, search, torrent, theme).
- `internal/config/` – Env config loading (`godotenv`).
- `internal/middleware/` – CORS + zerolog request logging.
- `Dockerfile` – multi-stage build.
- `scripts/build.sh` – local build script.

Current state: The Go backend is a skeleton/rewrite-in-progress. It replicates the API surface and proxies to qBittorrent's built-in search API, but lacks the plugin ecosystem, deduplication, enrichment, real download proxying, scheduled execution, and private-tracker auth flows that make the Python backend feature-complete.

Frontend (`frontend/`)

Angular 21 standalone application. Uses signals, RxJS, Angular CDK. Vitest for unit tests. Prettier for formatting. Built with `npx ng build` and served from the merge service.

- Output path: `../download-proxy/src/ui/dist/frontend`
- Production budgets: 500 kB warning / 1 MB error for initial bundle
- Coverage provider: v8; thresholds: 40% across lines, branches, functions, statements

Code Organization

```
download-proxy/src/
  main.py               # Entry: starts proxy + FastAPI in
threads
  api/
    __init__.py         # FastAPI app factory + middleware +
lifespan
    routes.py           # REST endpoints (search, download, auth,
theme, health)
    streaming.py        # SSE result streaming
    hooks.py            # Hook registration / invocation
    auth.py             # Auth state + credential management
    scheduler.py        # Scheduled search API
    theme_state.py      # Cross-app theme state (:7187 dashboard
only)
  merge_service/
    __init__.py         # Package init
    search.py           # Search orchestration + tracker parsing
    deduplicator.py     # Result deduplication logic
```

```
    enricher.py          # Metadata enrichment (OMDb, TMDb, etc.)
    validator.py         # Result validation
    scheduler.py         # Scheduled search engine
    hooks.py             # Merge-service hook firing
    retry.py             # Retry helpers
  config/
    __init__.py          # EnvConfig dataclass, env loading
    log_filter.py        # CredentialScrubber for logs
  ui/
    templates/           # Jinja2 dashboard + theme.css

qBitTorrent-go/          # Go/Gin backend (opt-in replacement)
  cmd/
    qbittorrent-proxy/   # Main API binary (port 7187)
    webui-bridge/        # Bridge binary (port 7188)
  internal/
    api/                 # HTTP handlers
    service/             # Merge search orchestrator + SSE broker
    client/              # qBittorrent Web API client
    models/              # Data types
    config/              # Env config
    middleware/          # CORS + logging
  Dockerfile
  scripts/build.sh

plugins/                 # Tracker plugins + support files
  *.py                   # Core plugins (eztv, jackett,
limetorrents, piratebay, ...)
  community/             # Additional community plugins
  webui_compatible/      # WebUI variants for private trackers
  helpers.py             # Shared plugin utilities
  nova2.py               # Nova search interface
  novaprinter.py         # Plugin output formatter
  socks.py               # SOCKS proxy support
  download_proxy.py      # Legacy HTTP proxy server (:7186)
  env_loader.py          # Dotenv loader used by some plugins

tests/                   # All tests (NOT in download-proxy/
tests/)
  unit/                  # Unit tests (heavily mocked)
    merge_service/       # Core logic tests
    api_layer/           # API layer tests
  contract/              # API contract tests (OpenAPI)
  property/              # Hypothesis property-based tests
  concurrency/           # Semaphore / concurrency tests
  memory/                # Memory leak tests (tracemalloc)
  observability/         # Prometheus metric assertions
  benchmark/             # Performance benchmarks
  chaos/                 # Fault-injection tests
  load/                  # Load tests
  performance/           # Performance regression tests
  stress/                # High-load stress tests
```

```
  security/              # Security-focused tests
  integration/           # Integration tests (needs running
containers or mocks)
  e2e/                   # End-to-end pipeline (needs running
containers)
  docs/                  # Documentation integrity (broken links)
  fixtures/              # Shared live-service fixtures
  conftest.py            # Global pytest config + fixtures

scripts/                 # Build / scan / test orchestration
scripts
  run-tests.sh           # Full suite with coverage (hermetic |
live | all)
  scan.sh                # Security + quality scanner
orchestration
  build-releases.sh      # Release artefact builder
  audit-plugins.sh       # Plugin audit
  freeze-openapi.sh      # OpenAPI spec freeze
  helixqa.sh             # HelixQA runner
  opencode-helixqa.sh    # OpenCode HelixQA runner
  add-submodules.sh      # Submodule setup

frontend/                # Angular 21 dashboard
  src/                   # TypeScript sources
  public/                # Static assets (PWA icons, manifest)
  package.json
  angular.json
  .prettierrc
  vitest.config.ts
```

Critical Constraints

- **TDD mandatory**: RED -> watch fail -> GREEN -> verify -> commit
- **Anti-Bluff Verification (CONST-XII)** — Tests and challenges MUST prove the feature works for the END USER. A green run is a contract with the user. Specifically:
  - Assert on user-observable outcomes (DB rows, file content, response body fields, container state, DOM text, rendered HTML attributes, browser console errors) — NOT just status codes / “no error”.
  - Each new test must fail against a no-op stub of the feature it tests. If it doesn’t, it’s a bluff and must be strengthened or deleted.
  - Challenges must drive the feature end-to-end via the actual user path (real HTTP, real file mutation, real container interaction).
  - For any new HTTP endpoint / CLI command / user-facing behavior: paste terminal output of an actual end-user

invocation in the same session as the change. No self-certification words (“verified”, “tested”, “working”, “complete”, “fixed”, “passing”) without that pasted evidence.
    - Flaky tests are bluffs; they must be hardened or deleted.
    - Regression tests MUST fail against the pre-fix code.
    - No hardcoded `localhost` / `127.0.0.1` for client-facing URLs. Any URL, API base, CORS origin, or service address returned to a browser MUST derive from the request’s `Host` header, `window.location`, or an explicit `PUBLIC_HOST` env var.
    - See `CONSTITUTION.md` § XII for the full rule. Apply universally — every submodule and sub-project inherits.
- **Never commit `.env`** – tracker credentials live there
- **WebUI credentials `admin`/`admin` are hardcoded** – do not change
- **IPTorrents freeleech only** – automated tests must never download non-freeleech. Tagged `IPTorrents [free]`
- **No comments in merge service source** (`download-proxy/src/`) – project convention
- **CI is manual via `./ci.sh`** – the canonical (and only) validation pipeline. All GitHub Actions workflow files have been removed per the Hard Stop rule. No automated CI/CD exists anywhere in the repository.

Key Commands

Startup

```
./setup.sh                              # One-time: dirs, config,
        plugins, containers
./start.sh                              # Start containers (-p
        pull, -s status, -v verbose)
./start.sh -p && python3 webui-bridge.py  # Full start with bridge
./stop.sh                               # Stop (-r remove, --purge
        clean images)
```

Go Backend (opt-in)

```
podman compose --profile go up -d      # Start Go backend instead
        of Python
cd qBitTorrent-go && ./scripts/build.sh   # Build Go binaries
        locally
cd qBitTorrent-go && go test -race ./...  # Go tests with race
        detection
cd qBitTorrent-go && go vet ./...         # Go static analysis
```

Testing

```
./ci.sh                                 # Full local CI: syntax +
        unit + integration + e2e + container health
./ci.sh --quick                         # Syntax + unit only
./ci.sh --tests-only                    # Skip syntax, run tests
scripts/run-tests.sh                    # Full suite with coverage
        (hermetic | live | all)
scripts/run-tests.sh hermetic           # Only hermetic suites
        (fast)
scripts/run-tests.sh live               # Integration + e2e only
        (slow, needs containers)
```

Single test / subset

```
python3 -m pytest tests/unit/test_freeleech.py -v --import-
        mode=importlib
python3 -m pytest tests/unit/merge_service/ -v --import-
        mode=importlib
python3 -m pytest tests/unit/ -k "search" -v --import-
        mode=importlib
```

Go Backend Tests

```
cd qBitTorrent-go && go test -race -count=1 ./...  # Full suite
        with race detection
cd qBitTorrent-go && go test ./internal/api/ -v     # Single
        package
cd qBitTorrent-go && go vet ./...                   # Static
        analysis
```

Lint + Typecheck

```
ruff check .                            # Lint (config in
        pyproject.toml)
ruff check --fix .
ruff format .
mypy download-proxy/src/                # Strict mypy (config in
        pyproject.toml)
```

Frontend

```
cd frontend && npm test                 # Vitest unit tests
        (interactive)
cd frontend && npx ng build             # Production build
cd frontend && npx vitest run           # Non-interactive Vitest
cd frontend && npm run test:coverage    # Coverage, single run
```

Sync code to container after edits

```
podman exec qbittorrent-proxy find /config/download-proxy -name
        __pycache__ -type d -exec rm -rf {} +
podman restart qbittorrent-proxy
```

For plugin edits: `./install-plugin.sh <name>` copies `plugins/X.py` -> `config/qBittorrent/nova3/engines/X.py` (source of truth is `plugins/`). Direct edits to the engines dir get clobbered on next install.

For compose/image changes: `podman compose down && podman compose up -d` (full recreate).

Always verify after restart: curl the endpoint or `podman exec ... cat /config/...` to confirm running code matches committed code.

Test Layout

Tests live in `./tests/`, NOT in `download-proxy/tests/`.

Directory	What	Requires
`tests/unit/`	Unit tests (heavily mocked)	Nothing
`tests/unit/merge_service/`	Core logic: dedup, search, hooks, validator, enricher, scheduler	Nothing
`tests/unit/api_layer/`	API layer unit tests	Nothing
`tests/contract/`	API contract tests (OpenAPI)	Nothing
`tests/property/`	Hypothesis property-based	Nothing
`tests/concurrency/`	Semaphore / concurrency	Nothing
`tests/memory/`	Memory leak (tracemalloc)	Nothing
`tests/observability/`		Nothing

Directory	What	Requires
	Prometheus metric assertions	
`tests/benchmark/`	Performance benchmarks	Nothing
`tests/chaos/`	Fault-injection / chaos	Nothing
`tests/performance/`	Performance regression tests	Nothing
`tests/stress/`	High-load stress tests	Nothing
`tests/security/`	Security-focused tests	Running containers
`tests/integration/`	Integration tests	Running containers or mocks
`tests/e2e/`	End-to-end pipeline	Running containers
`tests/docs/`	Documentation integrity (broken links)	Nothing
`tests/fixtures/`	Shared live-service fixtures (`services.py`, `live_search.py`)	–

Key fixtures in `tests/conftest.py`: `mock_qbittorrent_api`, `sample_search_result`, `sample_merged_result`, `qbittorrent_host/port/url`. Live fixtures in `tests/fixtures/services.py`: `merge_service_live`, `qbittorrent_live`, `webui_bridge_live`, `all_services_live`.

Unit test `sys.modules` isolation

`conftest.py` has an autouse fixture `_isolate_download_proxy_modules` that isolates `sys.modules` for tests under `tests/unit/` only (stub packages for `api`/`merge_service` would pollute other tests). Integration/e2e tests are excluded because they import real modules with live async references.

Event-loop cleanup

`conftest.py` has an autouse fixture `_cleanup_event_loop` that forces a clean asyncio loop after every test to prevent `Runner.run()` pollution (critical on Python 3.13).

pytest markers

Defined in `pyproject.toml`: - `requires_credentials` – needs real tracker credentials (`.env`) - `requires_compose` – needs the full docker-compose stack up - `slow` – takes longer than 1s (opt-in) - `stress` – high-load stress test (opt-in) - `security` – security-focused test - `contract` – API contract test (schemathesis / openapi) - `property` – property-based (hypothesis) test - `memory` – memory-leak / tracemalloc test - `chaos` – chaos / fault-injection test - `observability` – scrapes prometheus / metrics

Toolchain Config (all in `pyproject.toml`)

- **ruff**: target py312, line-length 120, select `E F W I UP B SIM RUF ASYNC S PT C4 TID`, ignore `E501 S101 S603 S607`
- **mypy**: strict, py312, excludes `plugins/` and `tests/` and `download-proxy/src/ui/`
- **pytest**: `--import-mode=importlib`, `--timeout=60`, `--strict-markers`, asyncio_mode=auto, asyncio_default_test_loop_scope=function
- **coverage**: sources `download-proxy/src` and `plugins`, `fail_under=49` (raised from 1% baseline, see `docs/COVERAGE_BASELINE.md`)
- **mutmut**: paths `download-proxy/src/`

Coverage baseline

Total coverage is **49%** (5,683 statements, 2,708 missing). The `fail_under` gate in `pyproject.toml` is set to `49`. When raising the gate, update `docs/COVERAGE_BASELINE.md` simultaneously. Unit tests only are used for the baseline; integration/e2e tests require running containers and are not included.

Quality Stack (opt-in)

`docker-compose.quality.yml` adds scanners and observability – never started by `./start.sh`.

```
podman compose -f docker-compose.yml -f docker-
        compose.quality.yml --profile quality up -d
scripts/scan.sh --all
```

Tool	Config File	Purpose
SonarQube	`sonar-project.properties`	Code quality + coverage
Snyk	`.snyk`	Dependency vulnerability scan
Semgrep	`.semgrep.yml`	Static analysis rules
Trivy	`.trivyignore`, `.trivyignore.yaml`	Container image scanning
Gitleaks	`.gitleaks.toml`	Secret detection
Prometheus	`observability/prometheus.yml`	Metrics collection
Grafana	`observability/dashboards/`	Dashboard visualization

All scanner reports land in `artifacts/scans/<timestamp>/` as SARIF. Mandatory waiver format: Finding ID / reason / expiry.

Security Considerations

- **Threat model**: WebUI is bound to host network; hardcoded `admin`/`admin` is intentional for trusted LAN. Operator must change before exposing. Reverse-proxy or VPN is the operator's responsibility.
- **Credential storage**: Env vars only. Priority: shell env -> `./.env` -> `~/.qbit.env` -> container env. `.env` is `rw-------` and gitignored.
- **Tracker sessions** (`_tracker_sessions`) are in-memory only and **Fernet-encrypted at rest** via `EncryptedSessionStore` (`merge_service/search.py`) — cookies never sit as plaintext in the cache. Key is per-process ephemeral by default; set `SESSION_ENCRYPTION_KEY` (urlsafe-base64 32-byte Fernet key) to pin it. Sessions still evaporate on container restart.
- **Credential scrubbing**: `CredentialScrubber` in `config/log_filter.py` redacts passwords/api keys from logs.
- **CORS**: Defaults to a **localhost allowlist** (not a wildcard); override via `ALLOWED_ORIGINS` (comma-separated). `*` remains accepted as an explicit opt-in but logs a security warning.
- **SSE streams**: Protected by the unguessable `search_id` UUID, plus an **opt-in per-search bearer token** (`stream_token` in the search response). Enable enforcement with `SSE_REQUIRE_TOKEN=1`; the stream endpoint then requires the token via `?token=` or

`Authorization: Bearer`. The dashboard always sends it, so enabling enforcement is seamless after a frontend rebuild.
- **Plugin isolation**: Public trackers run in subprocesses with a deadline timeout so a crashing plugin cannot crash the orchestrator.
- **IPTorrents freeleech policy** (Constitution Principle VIII): Automated tests and downloads MUST only use freeleech results. Freeleech results are tagged `IPTorrents [free]`. The deduplicator refuses to merge non-freeleech IPTorrents results with results from other trackers. `tests/unit/test_freeleech.py` guards the rule.
- **Gitleaks**: Allowlists `admin:admin` literal to prevent doc false positives.

Environment Variables

Priority: shell env -> `./.env` -> `~/.qbit.env` -> container env.

Key variables: - `RUTRACKER_USERNAME/PASSWORD` – RuTracker credentials - `KINOZAL_USERNAME/PASSWORD` – Kinozal credentials (falls back to `IPTORRENTS_*` if unset) - `NNMCLUB_COOKIES` – NNM-Club cookie-based auth - `NNMCLUB_USERNAME/PASSWORD` – NNM-Club fallback credentials - `IPTORRENTS_USERNAME/PASSWORD` – IPTorrents credentials - `RUTOR_USERNAME/PASSWORD` – **not consumed.** RuTor is a PUBLIC tracker with no login endpoint; it needs no authentication. If `RUTOR_USERNAME`/`RUTOR_PASSWORD` happen to be set in `.env`, they are ignored — the RuTor plugin searches and downloads anonymously. - `QBITTORRENT_DATA_DIR` – host download path (platform-aware default: `/mnt/DATA` on Linux, `$HOME/qbit-data` on macOS) - `MERGE_SERVICE_PORT` – default `7187` - `MERGE_SERVICE_HOST` – default `0.0.0.0` - `PROXY_PORT` – default `7186` - `BRIDGE_PORT` – default `7188` - `ALLOWED_ORIGINS` – CORS origins, comma-separated (default: localhost allowlist on :7187/:4200; `*` opt-in warns) - `SSE_REQUIRE_TOKEN` – set `1`/`true` to require the per-search SSE bearer token on the stream endpoint (default off) - `SESSION_ENCRYPTION_KEY` – urlsafe-base64 32-byte Fernet key to pin tracker-session at-rest encryption across restarts (default: per-process ephemeral) - `MAX_CONCURRENT_SEARCHES` – default `5` - `MAX_CONCURRENT_TRACKERS` – default `10` - `PUBLIC_TRACKER_DEADLINE_SECONDS` – default `15` - `DISABLE_THEME_INJECTION` – obsolete no-op in the Python stack (2026-09-01): the themed qBittorrent-WebUI overlay was removed and the stock vanilla WebUI is now served byte-for-byte. Still read by the Go backend. - `LOG_LEVEL` – default `INFO` - `OMDB_API_KEY`, `TMDB_API_KEY`, `TVDB_API_KEY` – metadata enrichment - `ANILIST_CLIENT_ID` – anime metadata - `JACKETT_API_KEY` – public tracker uplift (auto-discovered

from Jackett container config; manual override optional) - `JACKETT_INDEXER_MAP` – CSV `NAME:indexer_id` overrides for the autoconfig fuzzy matcher (legacy; prefer DB `indexer_map_overrides`) - `JACKETT_AUTOCONFIG_EXCLUDE` – CSV prefix denylist for env scan; default `QBITTORRENT,JACKETT,WEBUI,PROXY,MERGE,BRIDGE` - `BOBA_MASTER_KEY` – 32-byte hex AES-256-GCM key encrypting `tracker_credentials` in `/config/boba.db`. Auto-generated on first boot by `bootstrap.EnsureMasterKey` (and `start.sh ensure_boba_master_key`). **Loss = total credential loss.** See `docs/BOBA_DATABASE.md` § "Master key lifecycle" - `BOBA_DB_PATH` – SQLite path; default `/config/boba.db`. File mode forced to `0600` - `BOBA_ENV_PATH` – host `.env` path used for one-shot bootstrap import; default `/host-env/.env` - `SNYK_TOKEN`, `SONAR_TOKEN` – scanner tokens (opt-in)

Jackett auto-configuration (canonical: boba-jackett:7189): credentials, indexer-map overrides, and run history live in the encrypted SQLite system DB `config/boba.db`. The Angular dashboard at `http://localhost:7187/jackett` is the canonical edit surface. `.env` is imported once on first boot, never re-read after. The Python `/api/v1/jackett/autoconfig/last` endpoint was removed (replaced by Go `/api/v1/jackett/autoconfig/runs`). See `docs/JACKETT_INTEGRATION.md` § "Auto-Configuration" and `docs/BOBA_DATABASE.md`.

Plugin System

`plugins/` contains core tracker plugins, community plugins, and webUI-compatible variants.

Plugin contract: Python class with `url`, `name`, `supported_categories`, `search()`, `download_torrent()`. Output via `novaprinter.print()`.

Installed to `config/qBittorrent/nova3/engines/`.

```
./install-plugin.sh --all              # Install all
./install-plugin.sh rutracker rutor    # Install specific
./install-plugin.sh --verify           # Verify installation
```

Support files (`plugins/`): - `helpers.py` – shared utilities for plugins - `nova2.py` – nova search interface - `novaprinter.py` – output formatter - `socks.py` – SOCKS proxy support

Gotchas

- Private tracker tests need valid `.env` credentials; RuTracker may require browser-solved CAPTCHA (cookies expire)
- Kinozal credentials fall back to IPTorrents if unset
- NNMClub needs `NNMCLUB_COOKIES` in `.env` for live testing
- IPTorrents non-freeleech results never merge with other trackers; only `[free]` tagged ones merge
- RuTor is a PUBLIC tracker with **no login endpoint** – it needs no auth. Any `RUTOR_USERNAME`/`RUTOR_PASSWORD` in `.env` are NOT consumed
- `config/download-proxy/src/` is **gitignored** – do not commit copied source trees
- Plugin source of truth is `plugins/X.py`, not `config/qBittorrent/nova3/engines/X.py`
- `webui-bridge.py` runs on port **7188** (not 7186)
- Merge service tests live in `./tests/`, not `download-proxy/tests/`
- Proxy container runs `start-proxy.sh` which installs deps from `requirements.txt` at startup (including Levenshtein)
- Hooks file is at `/config/download-proxy/hooks.json` inside container
- Go backend is opt-in via `--profile go` – does not replace Python by default
- Go `qbittorrent-proxy` binary serves on 7187 (same as Python merge service); both cannot run simultaneously
- The `download-proxy` container mounts `./download-proxy` to `/config/download-proxy` so live edits on the host are reflected inside the container without rebuild
- `frontend/` is a standalone Angular 21 SPA built to `download-proxy/src/ui/dist/frontend` and served by the FastAPI merge service

Commit Style

Format: `<type>: <subject>` where type is `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`. Branch naming: `feature/your-feature-name`.

Universal Mandatory Constraints

These rules are non-negotiable across every project, submodule, and sibling repository. They are derived from the HelixAgent root `CLAUDE.md`. Each project MUST surface them in its own `CLAUDE.md`, `AGENTS.md`, and `CONSTITUTION.md`. Project-specific addenda are welcome but cannot weaken or override these.

Hard Stops (permanent, non-negotiable)

1. **NO CI/CD pipelines.** No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks either. All builds and tests run manually or via Makefile/ script targets.
2. **NO HTTPS for Git.** SSH URLs only (`git@github.com:…`, `git@gitlab.com:…`, etc.) for clones, fetches, pushes, and submodule updates. Including for public repos. SSH keys are configured on every service.
3. **NO manual container commands.** Container orchestration is owned by the project's binary/orchestrator (e.g. `make build` → `./bin/<app>`). Direct `docker`/`podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows. The orchestrator reads its configured `.env` and brings up everything.

Mandatory Development Standards

1. **100% Test Coverage.** Every component MUST have unit, integration, E2E, automation, security/penetration, and benchmark tests. No false positives. Mocks/stubs ONLY in unit tests; all other test types use real data and live services.
2. **Challenge Coverage.** Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success — validate actual behavior, not return codes.
3. **Real Data.** Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.
4. **Health & Observability.** Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.
5. **Documentation & Quality.** Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.

6. **Validation Before Release.** Pass the project's full validation suite (`make ci-validate-all`-equivalent) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).
7. **No Mocks or Stubs in Production.** Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations. Only unit tests may use mocks/stubs.
8. **Comprehensive Verification.** Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives in tests or challenges. Grep-only validation is NEVER sufficient.
9. **Resource Limits for Tests & Challenges (CRITICAL).** ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1` for `go test`. Container limits required. The host runs mission-critical processes — exceeding limits causes system crashes.
10. **Bugfix Documentation.** All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` (or the project's equivalent) with root cause analysis, affected files, fix description, and a link to the verification test/challenge.
11. **Real Infrastructure for All Non-Unit Tests.** Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`, equivalent for other languages). ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — MUST execute against the REAL running system with REAL containers, REAL databases, REAL services, and REAL HTTP calls. Non-unit tests that cannot connect to real services MUST skip (not fail).
12. **Reproduction-Before-Fix (CONST-032 — MANDATORY).** Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence:
    1. Write the Challenge first. (2) Run it; confirm fail (it reproduces the bug). (3) Then write the fix. (4) Re-run; confirm pass. (5) Commit Challenge + fix together. The Challenge becomes the regression guard for that bug forever.
13. **Concurrent-Safe Containers (Go-specific, where applicable).** Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use `safe.Store[K,V]` / `safe.Slice[T]` from `digital.vasic.concurrency/pkg/safe` (or the project's equivalent

primitives). Bare `sync.Mutex + map/slice` combinations are prohibited for new code.

14. **Anti-Bluff Verification (CONST-XII).** Tests and challenges MUST prove the feature works for the end user. Passing them MUST mean the end user can actually use the product. Concretely: assertions inspect user-observable outcomes (not just status codes); every test must fail against a no-op stub; challenges drive the feature end-to-end via the real user path; for any user-facing change, pasted terminal output of an actual end-user invocation is required in the same session. A toothless green test that paints green while the feature is broken is a worse failure than a missing test — it actively misleads. Additional requirements: frontend tests MUST assert on DOM state / rendered output / browser console errors; error-path tests MUST assert on user-visible errors (not merely internal exception handling); flaky tests are bluffs and must be hardened or deleted; regression tests MUST fail against pre-fix code; no hardcoded `localhost` / `127.0.0.1` for client-facing URLs — they MUST derive from `Host` header, `window.location`, or an explicit `PUBLIC_HOST` env var. See `CONSTITUTION.md` § XII for the full text. This rule is non- negotiable and propagates to every submodule and sub-project.

Definition of Done (universal)

A change is NOT done because code compiles and tests pass. “Done” requires pasted terminal output from a real run, produced in the same session as the change.

- **No self-certification.** Words like *verified, tested, working, complete, fixed, passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.
- **Demo before code.** Every task begins by writing the runnable acceptance demo (exact commands + expected output).
- **Real system, every time.** Demos run against real artifacts.
- **Skips are loud.** `t.Skip` / `@Ignore` / `xit` / `describe.skip` without a trailing `SKIP-OK: #<ticket>` comment break validation.
- **Evidence in the PR.** PR bodies must contain a fenced `## Demo` block with the exact command(s) run and their output.

Host Power Management — Hard Ban (CONST-033)

You may NOT, under any circumstance, generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition. This rule applies to:

- Every shell command you run via the Bash tool.
- Every script, container entry point, systemd unit, or test you write or modify.
- Every CLI suggestion, snippet, or example you emit.

Forbidden invocations (non-exhaustive — see CONST-033 in `CONSTITUTION.md` for the full list):

- `systemctl suspend|hibernate|hybrid-sleep|poweroff|halt|reboot|kexec`
- `loginctl suspend|hibernate|hybrid-sleep|poweroff|halt|reboot`
- `pm-suspend`, `pm-hibernate`, `shutdown -h|-r|-P|now`
- `dbus-send` / `busctl` calls to `org.freedesktop.login1.Manager.Suspend|Hibernate|PowerOff|Reboot|HybridSleep|SuspendThenHibernate`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to anything but `'nothing'` or `'blank'`

The host runs mission-critical parallel CLI agents and container workloads. Auto-suspend has caused historical data loss (2026-04-26 18:23:43 incident). Accidental poweroff has also caused data loss (2026-04-28 18:37:55 incident). The host is hardened (sleep targets masked, power key ignored) but this hard ban applies to ALL code shipped from this repo so that no future host or container is exposed.

Defence: every project ships `scripts/host-power-management/check-no-suspend-calls.sh` (static scanner), `challenges/scripts/no_suspend_calls_challenge.sh` (challenge wrapper), and `challenges/scripts/host_no_auto_poweroff_challenge.sh` (host state verification). All MUST be wired into the project's CI / `run_all_challenges.sh`.

Full background: `docs/HOST_POWER_MANAGEMENT.md` and `CONSTITUTION.md` (CONST-033).

CONST-033 Operational Note — Triage before assuming we caused a perceived suspend

Same triage as in `CLAUDE.md` — when the user reports "computer froze / suspended / logged me out", run this BEFORE proposing any fix:

1. `uptime` — discontinuous = real suspend; continuous ≥ alleged downtime = no suspend.
2. `journalctl -k --since "24 hours ago" | grep -iE "will suspend|systemd-suspend"` — zero = no systemd suspend. Also check `will power off` for CONST-034.
3. `journalctl -k --since "24 hours ago" | grep -iE "oom-kill|killed process"` — decode `oom_memcg`: `libpod-…` = container OOM (containment); `user@1000.service` = user-session OOM (looks like logout).
4. Both CONST-033 challenges must remain PASS.
5. Document findings in `docs/incidents/<date>-*.md`.

Common false positives (NOT CONST-033 violations): GNOME screen lock, GUI compositor stall under memory pressure, foreign container OOM at its own cgroup cap.

Container hygiene corollary (mandatory for every new compose service): `mem_limit`, `pids_limit`, `oom_score_adj: 500`.

Podman/Docker cannot suspend the host. Rootless podman has no power-management permission; cgroup OOM is containment, not host impact.

See `CONSTITUTION.md` § "CONST-033 Operational Note" and `docs/incidents/2026-04-27-perceived-host-suspension-investigation.md`.

§11.4.109 Anti-Forgetting Enforcement

Anti-forgetting enforcement is wired as a three-layer mechanical guard: - **PreToolUse hook** at `.claude/settings.json` → `constitution/scripts/hooks/guard-forbidden-commands.sh` blocks host-direct emulator, force-push, sudo, and host-power at the tool-call boundary. - **Subagent Constitutional Preamble** at `docs/AGENT_GUARDRAILS.md` — paste verbatim into every subagent dispatch. - **Orchestrator Pre-Action Checklist** at same file — consult before any dispatch/device/push/host-affecting action.

Hermetic test: `tests/hooks/test_guard_forbidden_commands.sh` (≥20 cases, all blocked classes + allowed + escape hatch + non-overridable host-power).

The hook script is inherited from the constitution submodule by reference — never copied locally.

Session Continuation

At start of every session: read `docs/CONTINUATION.md` first. It contains:

1. Current project state and last verification results
2. All known issues (honest, not swept)
3. Uncommitted changes list
4. Quick-start commands to bring up infrastructure
5. Architecture reference

When the user sends `continue`, pick up from `docs/CONTINUATION.md` as the authoritative handoff point.